

Introduction to Programming

Problem Sheet 3

```
{- HLINT ignore "Use map" -}  
import Data.Char (ord, chr)  
import Control.Monad (when)  
import Control.Arrow ((>>>))  
import Control.Exception (evaluate, handle, SomeException (SomeException))
```

Problem Sheet 3

PROBLEM 1

Recall that the identity function on lists can be defined using `foldr` in this fashion:

```
id' :: [a] -> [a]  
id' = foldr (:) []
```

Taking inspiration from the above, define the `map` function using `foldr`

```
map' :: (a -> b) -> [a] -> [b]  
map' = undefined
```

PROBLEM 2

Given a list of functions, which have the same domain and codomain, compose all of them to get a new function of the same type. For example, `compAll [f1, f2, f3] = f1 compose f2 compose f3` By convention, `compAll [] = id` Try to come up with a solution that uses `foldr`

```
compAll :: [a -> a] -> a -> a  
compAll = undefined
```

PROBLEM 3

Think of the lowercase letters 'a' to 'z' arranged on a circle, going from 'a' to 'z' in a clockwise fashion, and then going back from 'z' to 'a'. "Rotating" a letter by `n` steps means moving `n` steps on the circle starting from the given letter and seeing what letter you finally get.

Given a string consisting of all lowercase characters (and possibly spaces), `rotCipher n` rotates each letter by `n`, while not disturbing the spaces between them.

Remember that `ord 'a' = 97`, `ord 'z' = 122` and `ord ' ' = 32`.

```
rotCipher :: Int -> String -> String  
rotCipher = undefined
```

PROBLEM 4

Given a list of items, `chunkDivider n l` keeps taking out chunks of size `n` from the left side of the list `l` (till `l` is exhausted or the remaining list size is less than `n`). Just to clarify, break into chunk of size `n`, and ignore the excess leftover part of `l` if you cannot extract a chunk of size `n` from the left anymore. For example, `chunkDivider 2 [1 .. 11] = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]`

```
chunkDivider :: Int -> [a] -> [[a]]  
chunkDivider = undefined
```

PROBLEM 5

Implement a safe integer division function using `Maybe` where division by zero is handled gracefully by returning `Nothing`.

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv = undefined
```

PROBLEM 6

Implement a safe positive integer addition function using `Maybe`, where `Just n` is considered as the integer `n` (for $n > 0$), and `Nothing` is positive infinity. The convention that we are using is that adding anything to infinity gives infinity. You do not need to consider negative integers, or negative infinity.

```
safeAdd :: Maybe Int -> Maybe Int -> Maybe Int
safeAdd = undefined
```

The following functions are not part of the problem sheet. These are test cases we have provided so that you can self-check the functions you have written. Run the `main` function to see a summary of how your functions fare on the test cases. Note that successfully passing all test cases is not a guarantee that your program is correct, it merely shows that it passes all the tests that we have designed.